

Laporan Tugas Akhir

**“Implementasi Permintaan Vegapunk:
Sebuah Upaya Realisasi yang Terstruktur”**



Disusun Oleh Kelompok 04 2024 A:

Aida Zahira	(24031554106)
Fauziah Roikhana Wardah	(24031554162)
Ulfia Nurmalizha Ramadhani	(24031554064)

Dosen Pengampu:

Ibnu Febry Kurniawan, S.Kom., M.Sc., Ph.D.

Moh. Khoridatul Huda, S.Pd., M.Si., Ph.D.

**PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA**

2025

A. Deskripsi Program

1. Tahap Instalasi dan uv sync

Pada tahap instalasi, kami melakukannya pada terminal powershell yang tersedia. Setelah instalasi uv berhasil, akan dilanjutkan proses berikutnya yaitu uv sync. Perintah uv sync akan membuat sebuah virtual environment ``.venv`` (hidden) pada root folder, lalu memasang libraries yang dibutuhkan.

2. Melengkapi file python client

Pada tahap ini, dilakukan pengisian kode program untuk memenuhi TODO yang diminta. Dimana, pada file ini membuat private dan public key lalu disimpan untuk tahap selanjutnya. Setelah TODO pertama selesai, selanjutnya membuat pesan rahasia dan proses-proses lain enkripsi simetrik. Proses enkripsi menggunakan skema ECIES yang diawali dengan pembuatan kunci publik dan privat sementara (ephemeral key) berbasis kurva SECP256K1. Kunci privat sementara ini kemudian dikombinasikan dengan kunci publik server melalui protokol ECDH untuk menghasilkan rahasia bersama yang diperkuat menggunakan HKDF (SHA-256) menjadi kunci simetris 32-byte. Plain text selanjutnya dienkripsi menggunakan algoritma AES-GCM dengan tambahan nonce acak dan data autentikasi (ID pengirim dan penerima) untuk menjamin kerahasiaan sekaligus integritas data. Hasil akhir berupa teks terenkripsi, nonce, dan kunci publik sementara dikemas dalam format JSON berbasis Base64 agar siap dikirimkan secara aman ke server.

3. Melengkapi file python api

Pada tahap ini, membuat layanan Security Service berbasis FastAPI yang berfungsi sebagai pusat pengamanan data dan pesan. Pertama, mengatur sistem autentikasi menggunakan Bearer Token dan validasi daftar pengguna yang diizinkan (AUTHORIZED_USERS) untuk memastikan hanya orang tertentu yang bisa mengakses API ini. Untuk urusan penyimpanan, dibuat fungsi untuk mengunggah PDF dan kunci publik (ECDSA/Ed25519) yang sudah dilengkapi pemeriksaan integritas menggunakan SHA-256, jadi kalau file kuncinya rusak atau dimodifikasi saat dikirim, server bakal langsung tahu. Selain itu, ada fitur verifikasi tanda tangan digital yang secara otomatis mencocokkan pesan atau file PDF dengan kunci publik yang tersimpan untuk menjamin bahwa data tersebut asli dan belum diubah. Terakhir, menambahkan fitur relay pesan yang cukup kompleks, di sini server melakukan pertukaran kunci lewat protokol ECDH (melalui fungsi `derive_shared_key`) untuk menghasilkan kunci simetris, lalu mendekripsi pesan menggunakan AES-GCM sebelum menyimpannya, sehingga kerahasiaan pesan tetap terjaga aman sampai ke tujuan.

B. Contoh Penggunaan Program

1. Skor E simpan publik key(store)

```
@app.post("/store")
async def store_pubkey(
    username: str,
    pubkey: UploadFile = File(...)
):
    # pesan kembalian ke user (sukses/gagal)
    msg = None
    # Tuliskan code Anda di sini
    try:
        data = await pubkey.read()

        if b"PUBLIC KEY" not in data:
            raise HTTPException(status_code=400, detail="Invalid public key format")

        os.makedirs("pubkeys", exist_ok=True)
        with open(f"pubkeys/{username}.pem", "wb") as f:
            f.write(data)

        msg = "Public key stored successfully"

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    # Nilai kembalian berupa dictionary
    # Tambahkan keys dan values sesuai dengan kebutuhan
    return {
        "message": msg,
        "user": username
    }
```

kode tersebut berfungsi sebagai endpoint store, untuk menerima unggahan file public key, memvalidasi format isinya, dan menyimpannya ke dalam direktori server secara otomatis berdasarkan nama pengguna yang diberikan. Kegunaan utamanya adalah untuk mengelola sistem keamanan atau enkripsi data, di mana public key pengguna disimpan secara terpusat agar nantinya dapat digunakan untuk proses autentikasi atau verifikasi tanda tangan digital.

POST /store Store Pubkey

Parameters Cancel Reset

Name	Description
username * required string (query)	pub_client

Request body * required multipart/form-data

pubkey * required
string(\$binary) Choose File pub_client.pem

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/store?username=pub_client' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'pubkey=pub_client.pem'
```

Request URL

http://localhost:8080/store?username=pub_client

Server response

Code Details

200

Response body

```
{
  "message": "Public key stored successfully",
  "user": "pub_client"
}
```

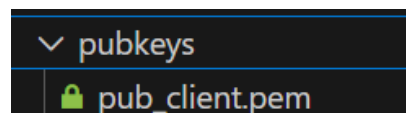
Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 64
content-type: application/json
date: Wed, 27 Dec 2023 15:11:53 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Antarmuka tersebut berfungsi untuk mendaftarkan kunci publik pengguna ke dalam sistem. Pengguna wajib mengisi parameter username agar server dapat menghubungkan kunci tersebut dengan identitas akun yang tepat, serta mengunggah file fisik kunci publik berformat .pem pada bagian Request body. Dengan mendaftarkan kunci ini, server dapat menyimpan basis data keamanan yang diperlukan untuk memverifikasi tanda tangan digital atau mengenkripsi komunikasi bagi pengguna tersebut di masa mendatang.



pubkey dapat dilihat pada file pub_client.pem

2. Skor E verify signature

```
@app.post("/verify")
async def verify(username: str, message: str, signature: str,
                 algo: str):
    # pesan kembalian ke user (sukses/gagal)
    msg = None
    valid = False
    # Tuliskan code Anda di sini
    try:
        with open(f"pubkeys/{username}.pem", "rb") as f:
            pubkey = serialization.load_pem_public_key(f.read())

        sig_bytes = base64.b64decode(signature)

        if algo == "ecdsa":
            pubkey.verify(sig_bytes, message.encode(),
                          ec.ECDSA(hashes.SHA256()))
        elif algo == "ed25519":
            pubkey.verify(sig_bytes,
                          message.encode())
        else:
            raise HTTPException(status_code=400,
                                detail="Unknown algorithm")

        msg = "Signature VALID"
        valid = True

    except Exception:
        msg = "Signature INVALID"
        valid = False
    # Nilai kembalian berupa dictionary
    # Tambahkan keys dan values sesuai dengan kebutuhan
    return {"message": msg, "user": username, "valid": valid}
```

Kode tersebut berfungsi sebagai endpoint verify untuk memverifikasi keaslian digital signature pengguna dengan mencocokkan pesan dan signature yang dikirim ke public key di server. Melalui algoritma kriptografi ECDSA atau Ed25519, kode tersebut memastikan integritas data serta validitas identitas pengirim untuk mencegah pemalsuan informasi dalam sistem komunikasi aman.

POST

/verify

Verify

Parameters

Cancel

Name	Description
username * required	
string (query)	pub_client
message * required	
string (query)	Diam - diam yaa ini pesan rahasia.
signature * required	
string (query)	MEYCIQDpiMnluQUsLbizD8AgRo9J7u/mx4i
algo * required	
string (query)	ecdsa

ExecuteClear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify?username=pub_client&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEYCIQDpiMnluQUsLbizD8AgRo9J7u/mx4i&algo=ecdsa' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8080/verify?username=pub_client&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEYCIQDpiMnluQUsLbizD8AgRo9J7u/mx4i&algo=ecdsa
```

Server response

Code	Details
200	Response body <pre>{ "message": "Signature VALID", "user": "pub_client", "valid": true }</pre> Download Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: * content-length: 62 content-type: application/json date: Wed, 17 Dec 2023 15:18:49 GMT server: uvicorn</pre>

Pada antarmuka Swagger, terdapat fitur /verify di mana pengguna harus memasukkan username, isi pesan asli (message), digital signature, serta algoritma yang digunakan (algo) seperti ecdsa untuk divalidasi oleh server. Berguna untuk menguji apakah digital signature valid atau tidak berdasarkan kunci publik yang sudah disimpan sebelumnya.

3. Skor E relay message

```
@app.post("/relay")
async def relay(sender: str, receiver: str, certificate: str,
               client_public_key: str, ephemeral_public_key: str,
               nonce: str, ciphertext: str,
               signature_ed25519: str, signature_ecdsa: str):
    msg = None
    success = False
    try:
        # Dekode data Base64
        eph_pub_bytes = base64.b64decode(ephemeral_public_key)
        eph_pub = serialization.load_pem_public_key(eph_pub_bytes)
        nonce_b = base64.b64decode(nonce)
        cipher_b = base64.b64decode(ciphertext)
        # Proses Derivasi Kunci & Dekripsi (Logika Inti)
        shared_key = derive_shared_key(SERVER_PRIVATE_KEY, eph_pub)
        aesgcm = AESGCM(shared_key)
        aad = f"{sender}|{receiver}".encode()
        plaintext = aesgcm.decrypt(nonce_b, cipher_b, aad)
        # Simpan pesan ke file
        os.makedirs("messages", exist_ok=True)
        with open(f"messages/{receiver}.txt", "a") as f:
            f.write(f"From {sender}: {plaintext.decode()}\n")

        msg = "Secure message relayed successfully"
        success = True

    except Exception as e:
        msg = f"Relay failed: {str(e)}"
        success = False
    # Nilai kembalian berupa dictionary
    # Tambahkan keys dan values sesuai dengan kebutuhan
    return {"message": msg, "from": sender,
           "to": receiver, "success": success}
```

Kode tersebut berfungsi untuk meneruskan pesan aman (relay) dengan cara mendekripsi pesan terenkripsi menggunakan protokol pertukaran kunci Diffie-Hellman dan algoritma AES-GCM, lalu menyimpan hasilnya ke dalam direktori server. Kegunaan utamanya adalah sebagai perantara komunikasi terenkripsi yang menjamin kerahasiaan data (aspek privacy) dan integritas pesan, sehingga hanya penerima yang dituju yang dapat mengakses isi pesan yang telah diproses oleh server tersebut.

POST /relay Relay

Parameters

Cancel

Name	Description
sender * required string (query)	client-ayam-kremes
receiver * required string (query)	server-punkhazard
certificate * required string (query)	LS0tLS1CRUdJTBDRVJUSUZJQ0FURS0tL
client_public_key * required string (query)	LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL
ephemeral_public_key * required string (query)	LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL
nonce * required string (query)	bdaJPFd8czR1P175
ciphertext * required string (query)	vi8V0QKEW1UlrChBAkgttuaqd5sCPbTwN8
signature_ed25519 * required string (query)	gsTtYXeYnlqOoaB0tRqwQ1GUwn+aVrvnCt
signature_ecdsa * required string (query)	MEQCIDSTLHA2yYEUwh132dGRgskmP5Ey

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/relay?sender=client-ayam-kremes&receiver=server-punkhazard&certificate=LS0tLS1CRUdJTBDRVJUSUZJQ0FURS0tL&client_public_key=LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL&ephemeral_public_key=LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL&nonce=bdaJPFd8czR1P175&ciphertext=vi8V0QKEW1UlrChBAkgttuaqd5sCPbTwN8&signature_ed25519=gsTtYXeYnlqOoaB0tRqwQ1GUwn+aVrvnCt&signature_ecdsa=MEQCIDSTLHA2yYEUwh132dGRgskmP5Ey' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8080/relay?sender=client-ayam-kremes&receiver=server-punkhazard&certificate=LS0tLS1CRUdJTBDRVJUSUZJQ0FURS0tL&client_public_key=LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL&ephemeral_public_key=LS0tLS1CRUdJTBQVUJMSUMgS0VZLS0tL&nonce=bdaJPFd8czR1P175&ciphertext=vi8V0QKEW1UlrChBAkgttuaqd5sCPbTwN8&signature_ed25519=gsTtYXeYnlqOoaB0tRqwQ1GUwn+aVrvnCt&signature_ecdsa=MEQCIDSTLHA2yYEUwh132dGRgskmP5Ey
```

Server response

Code

Details

200

Response body

```
{
  "message": "Secure message relayed successfully",
  "from": "client-ayam-kremes",
  "to": "server-punkhazard",
  "success": true
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 117
content-type: application/json
date: Wed, 17 Dec 2025 17:37:51 GMT
server: uvicorn
```

Pada endpoint API relay, antarmuka meminta pengguna memasukkan parameter lengkap dari hasil enkripsi klien, mulai dari sender, receiver, certificate, hingga ciphertext dan kedua jenis tanda tangan digital. Hal ini membuat server melakukan verifikasi identitas pengirim sekaligus mendekripsi pesan menggunakan kunci publik yang sudah disimpan sebelumnya. Dan untuk mengirimkan pesan rahasia yang sudah terenkripsi sepenuhnya.

4. Skor E tandatangan pdf

```
@app.post("/upload-pdf")
async def upload_pdf(file: UploadFile = File(...)):
    fname = file.filename
    ctype = file.content_type

    try:
        contents = await file.read()
        os.makedirs("uploads", exist_ok=True)
        with open(f"uploads/{fname}", "wb") as f:
            f.write(contents)
            # Lanjutkan dengan logika pemrograman yang dibutuhkan di sini
    except Exception as e:
        # Handle the exception and provide a meaningful response
        raise HTTPException(status_code=400, detail=f"Error: {str(e)}")
        return {"message": str(e)}

    return {
        "message": "File uploaded!",
        "content-type": ctype,
        "filename": fname
    }
```

Kode tersebut untuk menerima unggahan file dokumen, membacanya secara asinkron, dan menyimpannya ke dalam direktori server dengan nama file asli yang diberikan pengguna. Kegunaan utamanya adalah sebagai sistem penyimpanan file sederhana yang memungkinkan aplikasi web menerima data dari pengguna, sekaligus memberikan umpan balik instan mengenai status keberhasilan unggahan dan tipe konten file tersebut.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /upload-pdf
- Parameters:** No parameters.
- Request body:** multipart/form-data. A file named "Soal-UAS-KID25.pdf" is attached.
- Execute:** A blue button to execute the request.
- Responses:** A section showing the response details.
 - Request URL:** http://localhost:8080/upload-pdf
 - Server response:** 200
 - Response body:** {"message": "File uploaded!", "content-type": "application/pdf", "filename": "Soal-UAS-KID25.pdf"}
 - Response headers:** access-control-allow-credentials: true, access-control-allow-origin: *, content-length: 90, content-type: application/json, date: Wed, 17 Dec 2025 17:39:18 GMT, server: uvicorn

Antarmuka tersebut berfungsi untuk mengirimkan dokumen fisik ke server sebagai tahap awal dalam alur kerja digital. Pengguna mengunggah file dokumen, agar sistem dapat menyimpan dan memproses data tersebut lebih

lanjut sebelum dilakukan pembubuhan tanda tangan digital atau verifikasi oleh pihak lain.

5. Skor B- simpan public key static entry dan integrity check

```
@app.post("/store")
async def store_pubkey(username: str, expected_hash: str, pubkey: UploadFile = File(...)):
    # pesan kembalian ke user (sukses/gagal)
    msg = None
    status_success = False
    # --- LANGKAH 1: STATIC ENTRY ---
    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403,
                            detail=f"User '{username}' tidak terdaftar (Static Entry Violation)")
    # Tuliskan code Anda di sini
    try:
        data = await pubkey.read()
        # --- LANGKAH 2: INTEGRITY CHECK ---
        actual_hash = hashlib.sha256(data).hexdigest()
        # Perbaikan: Membandingkan actual dengan expected dari user
        if actual_hash != expected_hash:
            msg = "Integrity Check Gagal! File rusak atau dimodifikasi."
            status_success = False
        else:
            # Jika lolos Integrity, cek format
            if b"PUBLIC KEY" not in data:
                raise HTTPException(status_code=400, detail="Invalid public key format")
            # Simpan file
            os.makedirs("pubkeys", exist_ok=True)
            with open(f"pubkeys/{username}.pem", "wb") as f:
                f.write(data)

            msg = "Public key stored successfully with Integrity Check PASSED"
            status_success = True
    except HTTPException as he:
        raise he
    except Exception as e:
        msg = f"Error: {str(e)}"
        status_success = False
    # Nilai kembalian berupa dictionary
    # Tambahkan keys dan values sesuai dengan kebutuhan
    return {"message": msg, "user": username, "success": status_success,
            "hash_calculated": actual_hash if 'actual_hash' in locals() else None}
```

Kode tersebut berfungsi sebagai endpoint store untuk menyimpan kunci publik pengguna secara aman dengan memverifikasi otorisasi akun dan melakukan pemeriksaan integritas menggunakan hash SHA-256. Kegunaannya adalah untuk memastikan bahwa file yang diunggah tidak mengalami kerusakan atau modifikasi selama transmisi, sekaligus menjamin hanya pengguna terdaftar yang dapat memperbarui kunci publik mereka dalam sistem.

POST

/store Store Pubkey

Cancel

Reset

Parameters

Name	Description
username * required	
string (query)	client-ayam-kremes
expected_hash * required	
string (query)	ddb248a5362d34dbf033dc43e4bdcbb7c67f

Request body required

multipart/form-data

pubkey * required

string(\$binary)

Choose File pub_client.pem

Execute

Clear

Responses

Curl

curl -X 'POST' \n'http://localhost:8080/store?username=client-ayam-kremes&expected_hash=ddb248a5362d34dbf033dc43e4bdcbb7c67f8e245ae858d423cdc0' \n-H 'accept: application/json' \n-H 'Content-Type: multipart/form-data' \n-F 'pubkey=@pub_client.pem'

Request URL

http://localhost:8080/store?username=client-ayam-kremes&expected_hash=ddb248a5362d34dbf033dc43e4bdcbb7c67f8e245ae858d423cdc0

Server response

Code

Details

200

Response body

```
{
  "message": "Public key stored successfully with Integrity Check PASSED",
  "user": "client-ayam-kremes",
  "success": true,
  "hash_calculated": "ddb248a5362d34dbf033dc43e4bdcbb7c67f8e245ae858d423cdc0"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 200
content-type: application/json
date: Wed, 27 Dec 2023 18:21:08 GMT
server: uvicorn
```

Untuk menyimpan Public Key milik klien ke dalam server. expected_hash pada endpoint /store, kegunaannya adalah untuk Integritas Data. Server akan mengecek apakah file yang diterima sesuai dengan hash ini untuk memastikan file tidak rusak atau dimanipulasi saat pengiriman.

6. Skor B- verifikasi signature static entry dan integrity check

```
@app.post("/verify")
async def verify(username: str, message: str, signature: str,
                 algo: str, message_hash: str # Tambahkan ini untuk Integrity Check
):
    # --- LANGKAH 1: STATIC ENTRY ---
    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403,
                            detail=f"User '{username}' tidak memiliki izin akses.")
    # --- LANGKAH 2: INTEGRITY CHECK ---
    # Kita cek apakah isi message sesuai dengan hash yang dikirim
    calculated_hash = hashlib.sha256(message.encode()).hexdigest()
    if calculated_hash != message_hash:
        return {"message": "Integrity Check Gagal! Pesan telah dimodifikasi.",
                "valid": False}
    # pesan kembalian ke user (sukses/gagal)
    msg = None
    valid = False
    # Tuliskan code Anda di sini
    try:
        with open(f"pubkeys/{username}.pem", "rb") as f:
            pubkey = serialization.load_pem_public_key(f.read())

        sig_bytes = base64.b64decode(signature)

        if algo == "ecdsa":
            pubkey.verify(sig_bytes, message.encode(),
                          ec.ECDSA(hashlib.SHA256()))
        elif algo == "ed25519":
            pubkey.verify(sig_bytes, message.encode())
        else:
            raise HTTPException(status_code=400, detail="Unknown algorithm")

        msg = "Signature VALID"
        valid = True

    except Exception:
        msg = "Signature INVALID"
        valid = False
    # Nilai kembalian berupa dictionary
    # Tambahkan keys dan values sesuai dengan kebutuhan
    return {"message": msg, "user": username, "valid": valid,
            "integrity_check": "Success" if valid else "Failed"}
```

Kode tersebut berfungsi sebagai endpoint API yang melakukan verifikasi tanda tangan digital sekaligus pemeriksaan integritas pesan menggunakan hash SHA-256 dan algoritma kriptografi (ECDSA atau Ed25519). Kegunaan utamanya adalah untuk menjamin bahwa pesan yang diterima benar-benar berasal dari pengguna terdaftar (otentikasi) dan isinya tidak mengalami modifikasi selama proses pengiriman (integritas), sehingga sistem dapat mencegah aksi pemalsuan informasi atau serangan man-in-the-middle secara efektif.

POST /verify Ventry

Parameters

Cancel

Name	Description
username * required string (query)	client-ayam-kremes
message * required string (query)	Diam - diam yaa ini pesan rahasia.
signature * required string (query)	MEUCIEd1REQwQSU4cGjwsxWPyi5FY1t
algo * required string (query)	ecdsa
message_hash * required string (query)	742374f159eecd73cc25a29fea07ba57cfe28f

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify?username=client-ayam-kremes&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEUCIEd1REQwQSU4cGjwsxWPyi5FY1t&algo=ecdsa&message_hash=742374f159eecd73cc25a29fea07ba57cfe28f' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/verify?username=client-ayam-kremes&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEUCIEd1REQwQSU4cGjwsxWPyi5FY1t&algo=ecdsa&message_hash=742374f159eecd73cc25a29fea07ba57cfe28f
```

Server response

Code

Details

200

Response body

```
{
  "message": "Signature Valid",
  "user": "client-ayam-kremes",
  "valid": true,
  "integrity_check": "Success"
}
```

Response headers

```
access-control-allow-credentials: true
```

Endpoint ini digunakan oleh server untuk memastikan bahwa sebuah pesan benar-benar dikirim oleh pengirim yang sah (autentikasi) dan isinya tidak diubah di tengah jalan. Server akan menggunakan Public Key yang sudah didaftarkan sebelumnya (pada endpoint /store) untuk memvalidasi tanda tangan tersebut. message_hash merupakan Nilai hash (sidik jari digital) dari pesan tersebut. Server akan menghitung ulang hash dari message dan mencocokkannya dengan message_hash ini untuk memastikan teks pesan tidak berubah sedikit pun.

7. Skor B- relay message static entry dan integrity check

```
@app.post("/relay")
async def relay(sender: str, receiver: str, certificate: str,
               client_public_key: str, ephemeral_public_key: str,
               nonce: str, ciphertext: str, signature_ed25519: str,
               signature_ecdsa: str, ciphertext_hash: str):
    # --- LANGKAH 1: STATIC ENTRY ---
    # Cek apakah pengirim DAN penerima adalah user yang terdaftar
    if sender not in AUTHORIZED_USERS or receiver not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403,
                            detail="Sender atau Receiver tidak terdaftar (Static Entry Violation)")
    # --- LANGKAH 2: INTEGRITY CHECK ---
    # Verifikasi bahwa ciphertext yang diterima belum dimodifikasi
    # Kita hitung hash dari string ciphertext (Base64) yang diterima
    calculated_hash = hashlib.sha256(ciphertext.encode()).hexdigest()
    if calculated_hash != ciphertext_hash:
        return {"message": "Integrity Check Gagal! Ciphertext telah dimodifikasi atau rusak.",
                "success": False}

    msg = None
    success = False
    try:
        # Dekode data Base64
        eph_pub_bytes = base64.b64decode(ephemeral_public_key)
        eph_pub = serialization.load_pem_public_key(eph_pub_bytes)
        nonce_b = base64.b64decode(nonce)
        cipher_b = base64.b64decode(ciphertext)
        # Proses Derivasi Kunci & Dekripsi (Logika Inti)
        shared_key = derive_shared_key(SERVER_PRIVATE_KEY, eph_pub)
        aesgcm = AESGCM(shared_key)
        aad = f"{sender}|{receiver}".encode()
        plaintext = aesgcm.decrypt(nonce_b, cipher_b, aad)
        # Simpan pesan ke file
        os.makedirs("messages", exist_ok=True)
        with open(f"messages/{receiver}.txt", "a") as f:
            f.write(f"From {sender}: {plaintext.decode()}\n")

        msg = "Secure message relayed successfully"
        success = True

    except Exception as e:
        msg = f"Relay failed: {str(e)}"
        success = False

# Nilai kembalian berupa dictionary
# Tambahkan keys dan values sesuai dengan kebutuhan
return {"message": msg, "from": sender,
        "to": receiver, "success": success,
        "integrity": "Verified" if success else "Failed"}
```

Kode tersebut berfungsi sebagai endpoint API relay yang mengelola pengiriman pesan aman dengan mengintegrasikan verifikasi otorisasi pengguna, pemeriksaan integritas data menggunakan hash SHA-256, dan dekripsi asimetris melalui protokol Diffie-Hellman serta AES-GCM. Kegunaan utamanya adalah untuk menjamin kerahasiaan dan validitas pesan dalam sistem komunikasi terenkripsi, di mana server memastikan bahwa data tidak dimodifikasi selama transmisi dan hanya pengirim serta penerima terdaftar yang dapat memproses informasi tersebut.

Isi pesan dikirim dalam bentuk *Ciphertext* agar *server relay* hanya bisa meneruskan data tanpa mampu membaca isinya. Keaslian pengirim dijamin oleh lapisan keamanan ganda dari *Signature Ed25519* dan *Signature ECDSA*, sedangkan integritas data dipastikan melalui *Ciphertext Hash* yang memverifikasi bahwa pesan tidak rusak atau dimodifikasi selama proses transmisi.

8. Skor B+ variasi cipher dan multiuser simpan pubkey ecdsa

```
@app.post("/store")
async def store_pubkey(username: str, algo: str, expected_hash: str, pubkey: UploadFile = File(...)):
    if username not in AUTHORIZED_USERS:
        | raise HTTPException(status_code=403, detail=f"User '{username}' tidak terdaftar.")

    try:
        data = await pubkey.read()
        actual_hash = hashlib.sha256(data).hexdigest()
        if actual_hash != expected_hash:
            | return {"message": "Integrity Check Gagal! File rusak atau dimodifikasi.", "success": False}

        user_dir = f"pubkeys/{username}"
        os.makedirs(user_dir, exist_ok=True)
        fname = "pub.pem" if algo.lower() == "ecdsa" else "pub19.pem"
        with open(f"{user_dir}/{fname}", "wb") as f:
            | f.write(data)

        return {"message": "Public key stored successfully.", "user": username, "success": True, "hash_calculated": actual_hash}
    except Exception as e:
        | return {"message": f"Error: {str(e)}", "user": username, "success": False}
```

Kode tersebut berfungsi sebagai endpoint API untuk menyimpan kunci publik pengguna secara terorganisir dengan mengintegrasikan validasi otorisasi, pemeriksaan integritas melalui hash SHA-256, serta pengelompokan file berdasarkan algoritma kriptografi yang digunakan (ECDSA atau Ed25519). Kegunaan utamanya adalah untuk membangun infrastruktur kunci publik yang aman, di mana sistem menjamin bahwa hanya pengguna terdaftar yang dapat mengunggah kunci dan data yang disimpan tetap utuh tanpa modifikasi, sehingga siap digunakan untuk proses autentikasi atau enkripsi pesan di tahap berikutnya.

POST /store Store Pubkey

Parameters

Name	Description
username * required string (query)	client-ayam-kremes
algo * required string (query)	ecdsa
expected_hash * required string (query)	9d12d6a9519df500362803d93b9dca78dee11

Request body required multipart/form-data

pubkey * required
string(binary)

Execute **Clear**

Responses

Request URL
http://localhost:8080/store?username=client-ayam-kremes&algo=ecdsa&expected_hash=9d12d6a9519df500362803d93b9dca78dee11678c3cbb24457cf499b94b428a8

Server response

Code	Details
200	Response body <pre>{ "message": "Public key stored successfully.", "user": "client-ayam-kremes", "success": true, "hash_calculated": "9d12d6a9519df500362803d93b9dca78dee11678c3cbb24457cf499b94b428a8" }</pre>

Endpoint store tersebut berfungsi untuk mendaftarkan public key pengguna ke dalam server. Pengguna diwajibkan menyertakan identitas unik pada parameter username agar server dapat menghubungkan kunci tersebut ke pemilik yang tepat, serta menentukan jenis algoritma kriptografi pada parameter algo (seperti ECDSA) untuk memastikan kompatibilitas sistem. Untuk menjamin integritas data selama proses pengiriman, parameter expected_hash digunakan sebagai pembanding otomatis, server hanya akan menerima file kunci jika nilai hash-nya sesuai untuk mencegah manipulasi atau kerusakan data. Terakhir, pengguna harus mengunggah file fisik kunci publik berformat .pem melalui parameter pubkey, yang nantinya akan digunakan oleh server sebagai basis otentikasi aman.

9. Skor B+ variasi cipher dan multiuser simpan pubkey ed25519

```
@app.post("/store")
async def store_pubkey(username: str, algo: str, expected_hash: str, pubkey: UploadFile = File(...)):
    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403, detail=f"User '{username}' tidak terdaftar.")

    try:
        data = await pubkey.read()
        actual_hash = hashlib.sha256(data).hexdigest()
        if actual_hash != expected_hash:
            return {"message": "Integrity Check Gagal! File rusak atau dimodifikasi.", "success": False}

        user_dir = f"pubkeys/{username}"
        os.makedirs(user_dir, exist_ok=True)
        fname = "pub.pem" if algo.lower() == "ecdsa" else "pub19.pem"
        with open(f"{user_dir}/{fname}", "wb") as f:
            f.write(data)

        return {"message": "Public key stored successfully.", "user": username, "success": True, "hash_calculated": actual_hash}
    except Exception as e:
        return {"message": f"Error: {str(e)}", "user": username, "success": False}
```

Kode tersebut berfungsi sebagai endpoint API untuk menyimpan kunci publik pengguna secara terorganisir dengan mengintegrasikan validasi otorisasi,

pemeriksaan integritas melalui hash SHA-256, serta pengelompokan file berdasarkan algoritma kriptografi yang digunakan (ECDSA atau Ed25519). Kegunaan utamanya adalah untuk membangun infrastruktur kunci publik yang aman, di mana sistem menjamin bahwa hanya pengguna terdaftar yang dapat mengunggah kunci dan data yang disimpan tetap utuh tanpa modifikasi, sehingga siap digunakan untuk proses autentikasi atau enkripsi pesan di tahap berikutnya.

POST /store/storePubkey

Parameters

Name	Description
username * required	string (query)
algo * required	string (query)
expected_hash * required	string (query)

Request body required

multipart/form-data

pubkey * required

string (\$binary)

Choose File pub1.pem

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/store?username=server-punkhazard&algo=ed25519&expected_hash=31b9acdb50c262007878fa0203af7f34b1d06' \
  -H 'accept: application/json' \
  -H 'content-type: multipart/form-data' \
  -F 'pubkey=@pub1.pem'
```

Request URL

http://localhost:8080/store?username=server-punkhazard&algo=ed25519&expected_hash=31b9acdb50c262007878fa0203af7f34b1d06

Server response

Code Details

200

Response body

```
{
  "message": "Public key stored successfully.",
  "user": "server-punkhazard",
  "success": true,
  "hash_calculated": "31b9acdb50c262007878fa0203af7f34b1d06"
}
```

Download

Endpoint store tersebut berfungsi untuk mendaftarkan public key pengguna ke dalam server. Pengguna diwajibkan menyertakan identitas unik pada parameter username agar server dapat menghubungkan kunci tersebut ke pemilik yang tepat, serta menentukan jenis algoritma kriptografi pada parameter algo (seperti ED25519) untuk memastikan kompatibilitas sistem. Untuk menjamin integritas data selama proses pengiriman, parameter expected_hash digunakan sebagai pembanding otomatis, server hanya akan menerima file kunci jika nilai hash-nya sesuai untuk mencegah manipulasi atau kerusakan data. Terakhir, pengguna harus mengunggah file fisik kunci publik berformat .pem melalui parameter pubkey, yang nantinya akan digunakan oleh server sebagai basis otentikasi aman.

10. Skor B+ variasi cipher dan multi user verify pdf ecdsa

```
@app.post("/verify-pdf")
async def verify_pdf(username: str, algo: str, pdf_hash: str, signature_pdf: str):
    # Ambil public key sesuai user dan algoritma
    key_file = f"pubkeys/{username}/pub.pem" if algo=="ecdsa" else f"pubkeys/{username}/pub19.pem"
    with open(key_file, "rb") as f:
        pubkey = serialization.load_pem_public_key(f.read())

    sig_bytes = b64decode(signature_pdf)
    pdf_bytes = b64decode(pdf_hash)

    try:
        if algo == "ecdsa":
            pubkey.verify(sig_bytes, pdf_bytes, ec.ECDSA(hashes.SHA256()))
        elif algo == "ed25519":
            pubkey.verify(sig_bytes, pdf_bytes)
        else:
            return {"message": "Unknown algorithm", "valid": False}

        msg = "Signature VALID"
        valid = True
        return {"message": msg, "user": username, "valid": valid, "integrity_check": "Success" if valid else "Failed",
                "algo": algo, "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}
    except:
        return {"message": msg, "user": username, "valid": valid, "integrity_check": "Success" if valid else "Failed",
                "algo": algo, "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}
```

Kode tersebut berfungsi sebagai endpoint API yang melakukan verifikasi tanda tangan digital khusus untuk file PDF dengan mencocokkan hash dokumen dan tanda tangan yang dikirim terhadap kunci publik pengguna yang tersimpan di server. Kegunaan utamanya adalah untuk menjamin autentisitas dan integritas dokumen PDF, di mana sistem secara otomatis memilih kunci yang sesuai berdasarkan algoritma kriptografi (ECDSA atau Ed25519) untuk memastikan bahwa dokumen tersebut benar-benar ditandatangani oleh pemilik kunci dan isinya tidak mengalami perubahan ilegal.

POST /verify-pdf Verify Pdf

Parameters

Name	Description
username * required	client-ayam-kremes
algo * required	ecdsa
pdf_hash * required	CFTIXmUksELFusNaSdKloa+RQCnUrKEjv0
signature * required	MEUCIQDdaQ2RYLgn/GIApHipRjJgUJ2OK06

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify-pdf?username=client-ayam-kremes&algo=ecdsa&pdf_hash=CFTIXmUksELFusNaSdKloa+RQCnUrKEjv0&signature=MEUCIQDdaQ2RYLgn/GIApHipRjJgUJ2OK06' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/verify-pdf?username=client-ayam-kremes&algo=ecdsa&pdf_hash=CFTIXmUksELFusNaSdKloa+RQCnUrKEjv0&signature=MEUCIQDdaQ2RYLgn/GIApHipRjJgUJ2OK06

Server response

Code Details

200

Response body

```
{
  "message": "Signature PDF VALID",
  "user": "client-ayam-kremes",
  "algo": "ecdsa",
  "success": true
}
```

Response headers

Endpoint verify-pdf berfungsi untuk memvalidasi keaslian dokumen PDF yang telah ditandatangani secara digital. Pengguna wajib menyertakan username agar server dapat menemukan kunci publik yang sesuai untuk

mencocokkan identitas, serta menentukan parameter algo (ECDSA) untuk memastikan algoritma kriptografi yang digunakan dalam proses verifikasi konsisten dengan saat kunci dibuat. Untuk menjamin integritas data, parameter pdf_hash dikirimkan sebagai sidik jari digital unik dari dokumen PDF, sementara parameter signature digunakan sebagai bukti otentikasi yang dibuat dengan kunci pribadi pengguna. Dengan kombinasi seluruh parameter tersebut, server dapat memastikan secara efektif bahwa dokumen tersebut benar-benar berasal dari pengirim yang sah dan isinya belum mengalami perubahan.

11. Skor B+ variasi cipher dan multi user verify pdf ed25519

```
@app.post("/verify-pdf")
async def verify_pdf(username: str, algo: str, pdf_hash: str, signature_pdf: str):
    # Ambil public key sesuai user dan algoritma
    key_file = f"pubkeys/{username}/pub.pem" if algo=="ecdsa" else f"pubkeys/{username}/pub19.pem"
    with open(key_file, "rb") as f:
        pubkey = serialization.load_pem_public_key(f.read())

    sig_bytes = b64decode(signature_pdf)
    pdf_bytes = b64decode(pdf_hash)

    try:
        if algo == "ecdsa":
            pubkey.verify(sig_bytes, pdf_bytes, ec.ECDSA(hashes.SHA256()))
        elif algo == "ed25519":
            pubkey.verify(sig_bytes, pdf_bytes)
        else:
            return {"message": "Unknown algorithm", "valid": False}

        msg = "Signature VALID"
        valid = True
        return {"message": msg, "user": username, "valid": valid, "integrity_check": "Success" if valid else "Failed",
                "algo": algo, "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}
    except:
        return {"message": msg, "user": username, "valid": valid, "integrity_check": "Success" if valid else "Failed",
                "algo": algo, "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}
```

Kode tersebut berfungsi sebagai endpoint API yang melakukan verifikasi tanda tangan digital khusus untuk file PDF dengan mencocokkan hash dokumen dan tanda tangan yang dikirim terhadap kunci publik pengguna yang tersimpan di server. Kegunaan utamanya adalah untuk menjamin autentisitas dan integritas dokumen PDF, di mana sistem secara otomatis memilih kunci yang sesuai berdasarkan algoritma kriptografi (ECDSA atau Ed25519) untuk memastikan bahwa dokumen tersebut benar-benar ditandatangani oleh pemilik kunci dan isinya tidak mengalami perubahan ilegal.

POST
/verify-pdf
Verify Pdf

Parameters
Cancel

Name	Description
username * required string (query)	server-punkhazard
algo * required string (query)	ed25519
pdf_hash * required string (query)	CFTXmUksELFusNaSdKloa+RQCnUrKEjv0
signature_pdf * required string (query)	sUsYv0/sEZFr9UblOYJB81TFVSqBAzfsx6

Execute
Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify-pdf?username=server-punkhazard&algo=ed25519&pdf_hash=CFTXmUksELFusNaSdKloa+RQCnUrKEjv0&signature_pdf=sUsYv0/sEZFr9UblOYJB81TFVSqBAzfsx6&_qfpu22AVXKTeACX2BUpY60H67s2WV2bu93u35dH6w430K3D' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/verify-pdf?username=server-punkhazard&algo=ed25519&pdf_hash=CFTXmUksELFusNaSdKloa+RQCnUrKEjv0&signature_pdf=sUsYv0/sEZFr9UblOYJB81TFVSqBAzfsx6&_qfpu22AVXKTeACX2BUpY60H67s2WV2bu93u35dH6w430K3D
```

Server response

Code
Details

200
Response body

```
{
  "message": "Signature VALID",
  "user": "server-punkhazard",
  "valid": true,
  "integrity_check": "Success",
  "algo": "ed25519",
  "calculated_hash": "7beba7b944cf52ab813e2ade7b2f954346fc4f29c8eac1288f8541d110b49068"
}
```

Download

Endpoint verify-pdf berfungsi untuk memvalidasi keaslian dokumen PDF yang telah ditandatangani secara digital. Pengguna wajib menyertakan username agar server dapat menemukan kunci publik yang sesuai untuk mencocokkan identitas, serta menentukan parameter algo (ED25519) untuk memastikan algoritma kriptografi yang digunakan dalam proses verifikasi konsisten dengan saat kunci dibuat. Untuk menjamin integritas data, parameter pdf_hash dikirimkan sebagai sidik jari digital unik dari dokumen PDF, sementara parameter signature digunakan sebagai bukti otentikasi yang dibuat dengan kunci pribadi pengguna. Dengan kombinasi seluruh parameter tersebut, server dapat memastikan secara efektif bahwa dokumen tersebut benar-benar berasal dari pengirim yang sah dan isinya belum mengalami perubahan.

12. Skor B+ variasi cipher dan multi user verify ecdsa

```
@app.post("/verify")
async def verify(username: str, message: str, signature: str, algo: str, message_hash: str):
    # --- LANGKAH 1: STATIC ENTRY / multiuser check ---
    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403, detail=f"User '{username}' tidak memiliki izin akses.")

    # --- LANGKAH 2: INTEGRITY CHECK ---
    calculated_hash = hashlib.sha256(message.encode()).hexdigest()
    if calculated_hash != message_hash:
        return {"message": "Integrity Check Gagal! Pesan telah dimodifikasi.",
                "user": username, "valid": False, "integrity_check": "Failed",
                "calculated_hash": calculated_hash}

    # --- LANGKAH 3: VERIFIKASI SIGNATURE ---
    valid = False
    msg = None
    try:
        # Pilih public key sesuai algoritma
        key_file = f"pubkeys/{username}/pub.pem" if algo.lower() == "ecdsa" else f"pubkeys/{username}/pub19.pem"
        with open(key_file, "rb") as f:
            pubkey = serialization.load_pem_public_key(f.read())

        sig_bytes = base64.b64decode(signature)

        # Verifikasi signature sesuai algoritma
        if algo.lower() == "ecdsa":
            pubkey.verify(sig_bytes, message.encode(), ec.ECDSA(hashes.SHA256()))
        elif algo.lower() == "ed25519":
            pubkey.verify(sig_bytes, message.encode())
        else:
            return {"message": "Unknown algorithm", "user": username, "valid": False}

        msg = "Signature VALID"
        valid = True
    except Exception:
        msg = "Signature INVALID"
        valid = False

    # --- LANGKAH 4: KEMBALIKAN RESPONSE ---
    return {"message": msg, "user": username, "valid": valid, "algo": algo.lower(),
            "integrity_check": "Success" if valid else "Failed", "calculated_hash": calculated_hash}
```

Kode tersebut berfungsi untuk memvalidasi identitas pengguna melalui skema verifikasi multi-langkah, yang mencakup pemeriksaan otorisasi akses, pengujian integritas pesan menggunakan hash SHA-256, serta verifikasi tanda tangan digital asimetris (ECDSA atau Ed25519). Kegunaan utamanya adalah sebagai sistem gerbang keamanan terpusat yang memastikan bahwa setiap data yang masuk tidak hanya berasal dari entitas terpercaya dan sah, tetapi juga terjamin keutuhannya tanpa mengalami modifikasi ilegal, sehingga menciptakan lingkungan pertukaran informasi yang memiliki aspek non-repudiation dan keamanan tinggi.

POST

/verify Verify

Parameters

Cancel

Name	Description
username * required string (query)	client-ayam-kremes
message * required string (query)	Diam - diam yaa ini pesan rahasia.
signature * required string (query)	MEUCIFWkrTNgl49FXEUF1d8eAxWJe0yud
algo * required string (query)	ecdsa
message_hash * required string (query)	742374f159eecd73cc25a29fea07ba57cfe28

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify?username=client-ayam-kremes&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEUCIFWkrTNgl49FXEUF1d8eAxWJe0yud&algo=ecdsa&message_hash=742374f159eecd73cc25a29fea07ba57cfe28' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8080/verify?username=client-ayam-kremes&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=MEUCIFWkrTNgl49FXEUF1d8eAxWJe0yud&algo=ecdsa&message_hash=742374f159eecd73cc25a29fea07ba57cfe28&accept=application/json
```

Server response

Code	Details
200	Response body <pre>{ "message": "Signature VALID", "user": "client-ayam-kremes", "valid": true, "algo": "ecdsa", "integrity_check": "Success", "calculated_hash": "742374f159eecd73cc25a29fea07ba57cfe2858c6d743789f6dde36121b1f71a" }</pre> Download

Response headers

Endpoint verify berfungsi untuk memvalidasi keaslian dan integritas pesan teks. Pengguna wajib menyertakan identitas pada parameter username agar server dapat mencocokkan kunci publik yang sesuai, serta menentukan parameter algo (seperti ECDSA) untuk memastikan konsistensi algoritma kriptografi yang digunakan. Dalam proses ini, parameter message_hash berperan sebagai sidik jari digital untuk memvalidasi isi pesan, sedangkan parameter signature berfungsi sebagai bukti otentikasi bahwa pesan tersebut benar-benar dikirim oleh pengguna yang sah. Dengan integrasi seluruh parameter tersebut, server dapat memastikan secara efektif bahwa pesan teks belum mengalami modifikasi dan berasal dari sumber yang terverifikasi.

13. Skor B+ variasi cipher dan multi user verify ed25519

```
@app.post("/verify")
async def verify(username: str, message: str, signature: str, algo: str, message_hash: str):
    # --- LANGKAH 1: STATIC ENTRY / multiuser check ---
    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403, detail=f"User '{username}' tidak memiliki izin akses.")

    # --- LANGKAH 2: INTEGRITY CHECK ---
    calculated_hash = hashlib.sha256(message.encode()).hexdigest()
    if calculated_hash != message_hash:
        return {"message": "Integrity Check Gagal! Pesan telah dimodifikasi.",
                "user": username, "valid": False, "integrity_check": "Failed",
                "calculated_hash": calculated_hash}

    # --- LANGKAH 3: VERIFIKASI SIGNATURE ---
    valid = False
    msg = None
    try:
        # Pilih public key sesuai algoritma
        key_file = f"pubkeys/{username}/pub.pem" if algo.lower() == "ecdsa" else f"pubkeys/{username}/pub19.pem"
        with open(key_file, "rb") as f:
            pubkey = serialization.load_pem_public_key(f.read())

        sig_bytes = base64.b64decode(signature)

        # Verifikasi signature sesuai algoritma
        if algo.lower() == "ecdsa":
            pubkey.verify(sig_bytes, message.encode(), ec.ECDSA(hashes.SHA256()))
        elif algo.lower() == "ed25519":
            pubkey.verify(sig_bytes, message.encode())
        else:
            return {"message": "Unknown algorithm", "user": username, "valid": False}

        msg = "Signature VALID"
        valid = True
    except Exception:
        msg = "Signature INVALID"
        valid = False

    # --- LANGKAH 4: KEMBALIKAN RESPONSE ---
    return {"message": msg, "user": username, "valid": valid, "algo": algo.lower(),
            "integrity_check": "Success" if valid else "Failed", "calculated_hash": calculated_hash}
```

Kode tersebut berfungsi untuk memvalidasi identitas pengguna melalui skema verifikasi multi-langkah, yang mencakup pemeriksaan otorisasi akses, pengujian integritas pesan menggunakan hash SHA-256, serta verifikasi tanda tangan digital asimetris (ECDSA atau Ed25519). Kegunaan utamanya adalah sebagai sistem gerbang keamanan terpusat yang memastikan bahwa setiap data yang masuk tidak hanya berasal dari entitas terpercaya dan sah, tetapi juga terjamin keutuhannya tanpa mengalami modifikasi ilegal, sehingga menciptakan lingkungan pertukaran informasi yang memiliki aspek non-repudiation dan keamanan tinggi.

POST

/verify Verify

Parameters

Cancel

Name	Description
username * required	
string (query)	server-punkhazard
message * required	
string (query)	Diam - diam yaa ini pesan rahasia.
signature * required	
string (query)	g8Xl+wyFV17vS0+MP3vVJvy2nji0I4SjfnGNl
algo * required	
string (query)	ed25519
message_hash * required	
string (query)	742374f159eecd73cc25a29fea07ba57cfe28f

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/verify?username=server-punkhazard&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=g8Xl%2BwyFV17vS0%2BMP3vVJvy2nji0I4SjfnGNl&algo=ed25519&message_hash=742374f159eecd73cc25a29fea07ba57cfe28f' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8080/verify?username=server-punkhazard&message=Diam%20-%20diam%20yaa%20ini%20pesan%20rahasia.&signature=g8Xl%2BwyFV17vS0%2BMP3vVJvy2nji0I4SjfnGNl&algo=ed25519&message_hash=742374f159eecd73cc25a29fea07ba57cfe28f
```

Server response

Code

Details

200

Response body

```
{
  "message": "Signature VALID",
  "user": "server-punkhazard",
  "valid": true,
  "algo": "ed25519",
  "integrity_check": "Success",
  "calculated_hash": "742374f159eecd73cc25a29fea07ba57cfe28f"
}
```

Download

Response headers

Endpoint verify berfungsi untuk memvalidasi keaslian dan integritas pesan teks. Pengguna wajib menyertakan identitas pada parameter username agar server dapat mencocokkan kunci publik yang sesuai, serta menentukan parameter algo (seperti ED25519) untuk memastikan konsistensi algoritma kriptografi yang digunakan. Dalam proses ini, parameter message_hash berperan sebagai sidik jari digital untuk memvalidasi isi pesan, sedangkan parameter signature berfungsi sebagai bukti otentikasi bahwa pesan tersebut benar-benar dikirim oleh pengguna yang sah. Dengan integrasi seluruh parameter tersebut, server dapat memastikan secara efektif bahwa pesan teks belum mengalami modifikasi dan berasal dari sumber yang terverifikasi.

14. Skor A Login

POST

/login Login

Parameters

Cancel

Name	Description
username required	
string	client-ayam-kremes
(query)	

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  "http://localhost:8080/login?username=client-ayam-kremes" \  
  -H 'accept: application/json' \  
  -d ''
```

Request URL

http://localhost:8080/login?username=client-ayam-kremes

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjbGllbnQVYXN1bS1rcmVtZDh1C3Rlc2VhIiwiaWF0IjE3MjY4NDUzMTI0.1jw74hwkcEBEzAfshdtq583pQoQW4Pw4x9KY1V4", "token_type": "Bearer" }</pre> Download

Endpoint login digunakan sebagai autentikasi awal bagi pengguna sebelum mengakses layanan. Proses ini menggunakan mengirimkan parameter username melalui HTTP POST, server melakukan validasi username dan menghasilkan JWT sebagai bukti pengguna telah diautentikasi. Token dikembalikan dalam bentuk access token dengan tipe bearer.

15. Skor A Authorize

Tampilan HTTP Bearer menunjukkan bahwa proses autentikasi menggunakan skema bearer token telah berhasil dilakukan. token dimasukkan ke authorize pada swagger, ketika status berubah menjadi authorized berarti token JWT sudah tersimpan dan secara otomatis akan dikirim pada setiap permintaan ke endpoint yang membutuhkan. Tombol logout digunakan untuk menghapus token dari sesi swagger apabila ingin mengganti pengguna atau mengakhiri autentikasi.

16. Skor A Setelah Input Token

POST	/store	Store Pubkey	🔒	▼
POST	/verify	Verify Signature	🔒	▼
POST	/sign-pdf	Sign Pdf	🔒	▼
POST	/relay	Relay	🔒	▼

Setelah proses login dan authorize berhasil endpoint store, verify, sign dan relay akan ditandai dengan ikon gembok pada swagger yang menunjukkan bahwa akses hanya diberikan kepada pengguna yang telah berhasil login dan memiliki token yang valid.

17. Skor A Store Pubkey

POST /store Store Data

Parameters

Cancel

Reset

Name	Description
username * required string(query)	
algo * required string(query)	
expected_hash * required string(query)	
Request body * required	multipart/form-data ▼
pubkey * required string(\$binary)	Choose File pub.pem

Execute

Clear

Responses

Curl

```
curl -X POST \
http://localhost:8080/store?username=client-ayam-kremes&algo=ecdsa&expected_hash=9eeef3e174687d32999f9eaeb1e0139fb633e347b484e821ca293bebf3a28bd \
-H "accept: application/json" \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTZlcnVycyBjbHkiLCJmYWlkIDJeImYwNDQMTTItLjE4OTkxMDEtZW5kaWQiOgEBAFohudqS8jpOpKwApPwhdcRY1LA." \
-H "Content-Type: multipart/form-data" \
-F "pubkey=@pub.pem"
```

Request URL

http://localhost:8080/store?username=client-ayam-kremes&algo=ecdsa&expected_hash=9eeef3e174687d32999f9eaeb1e0139fb633e347b484e821ca293bebf3a28bd

Server response

Code	Details
200	Response body<pre>{ "message": "Public key stored!", "user": "client-ayam-kremes", "algorithm": "ecdsa", "integrity": "fa53fd" }</pre> Download

18. Skor A Verify Signature

POST

/verify Verify Signature

Parameters

Name	Description
username * required string (query)	client-ayam-kremes
message * required string (body)	Diam-diam ini pesan rahasia.
signature * required string (header)	MEUCIQDbNow6Y3o3wfrf0LJKtEXI9Ve4g
algo * required string (header)	ecdsa
message_hash * required string (query)	25d390b806f669d376dc8b0684d5e58fad73

Execute

Clear

Responses

Curl

```
curl -X POST \\\nhttp://localhost:8080/verify?username=client-ayam-kremes&message=Diam-diam%2Bini%2Bpesan%2Brahasia.&signature=MEUCIQDbNow6Y3o3wfrf0LJKtEXI9Ve4g&sipbbZXBURrNhbSwJglEofLIjd237pyZY6cPUzjwrZ28hgGdhVa\\\n-w'accept: application/json'\\\n-Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5bGEiOiJSemludGVzLnVzcyIsImF1dG8iOiJkZW5pdWkiLCJhdWQiOiJkaWQyMDE5MDQzMTh3Iiwibm9ja3VzZXI6ImNoYS5kb3RvdmlldGEtcmVkdW8iLCJlbmdIdjE3bm9keSBMTTItLS1jcw74dwkcCBEtaHphdmg83p0xOWM4PhwcKXIyIA'\\\n-d''
```

<

>

Request URL

```
http://localhost:8080/verify?username=client-ayam-kremes&message=Diam-diam%2Bini%2Bpesan%2Brahasia.&signature=MEUCIQDbNow6Y3o3wfrf0LJKtEXI9Ve4g&sipbbZXBURrNhbSwJglEofLIjd237pyZY6cPUzjwrZ28hgGdhVa76zhTSXAkgncdsasmessage_hash=25d390b806f669d376dc8b0684d5e58fad73
```

Server response

Code	Details
200	Response body <pre>{\n "message": "Signature VALID",\n "valid": true\n}</pre> Download

19. Skor A Sign PDF

POST

/sign-pdf

sign-pdf

Parameters

Cancel

Reset

Name	Description
username * required string (query)	client-ayam-kremes
algo * required string (query)	ecdsa

Request body required

multipart/form-data

pdf * required
string(\$binary)

Choose File

Soal-UAS-KID25.pdf

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  "http://localhost:8080/sign-pdf?username=client-ayam-kremes&algo=ecdsa" \  
  -H 'accept: application/json' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC99.eyJzdWIiOiJjbGllbnQ6YXNjaW51cmwzLjZ0bGUzIiwiaWF0IjE5MTQ5MDUwMTI5. -LjA7w7hWkCEBzAFnhUq583pQgRW4Pw4wK1YIA' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'pdf=@Soal-UAS-KID25.pdf;type=application/pdf'
```

Request URL

http://localhost:8080/sign-pdf?username=client-ayam-kremes&algo=ecdsa

Server response

Code	Details
200	Response body <pre>{ "message": "PDF signed", "algorithm": "ecdsa", "signature_file": "signed-pdf/client-ayam-kremes_Soal-UAS-KID25.pdf.sig" }</pre> Download

20. Skor A Relay

[illegible]

C. Penjelasan Fungsi - Fungsi Dalam Program

1. Tahap Instalasi

Dilakukan instalasi uv terlebih dahulu sebelum memulai tahap berikutnya guna memastikan manajemen paket dan lingkungan virtual berjalan dengan cepat dan efisien.

```
PS C:\Users\DELL> powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
Downloading uv 0.9.18 (x86_64-pc-windows-msvc)
Installing to C:\Users\DELL\.local\bin
uv.exe
uvx.exe
uvw.exe
everything's installed!
```

Gambar 1. Instalasi

Server mengunduh berkas biner versi 0.9.18 dan memasang tiga komponen utama, yaitu uv.exe, uvx.exe, dan uvw.exe, ke dalam direktori lokal agar dapat digunakan untuk mengelola lingkungan pengembangan Python dengan lebih efisien.

```
PS C:\Users\DELL\Downloads\kid-uas> uv sync
Using CPython 3.10.19
Creating virtual environment at: .venv
Resolved 20 packages in 21ms
Installed 19 packages in 520ms
+ annotated-types==0.7.0
+ anyio==3.7.1
+ cffi==2.0.0
+ click==8.3.1
+ colorama==0.4.6
+ cryptography==46.0.3
+ exceptiongroup==1.3.1
+ fastapi==0.104.1
+ h11==0.16.0
+ idna==3.11
+ pycparser==2.23
+ pydantic==2.5.0
+ pydantic-core==2.14.1
+ python-dotenv==1.0.0
+ pydantic-core==2.14.1
+ pydantic-core==2.14.1
+ pydantic-core==2.14.1
+ pydantic-core==2.14.1
+ pydantic-core==2.14.1
+ pydantic-core==2.14.1
+ python-dotenv==1.0.0
+ python-multipart==0.0.20
+ sniffio==1.3.1
+ starlette==0.27.0
+ typing-extensions==4.15.0
+ uvicorn==0.24.0
```

Gambar 2. Output perintah uv sync

Perintah uv sync untuk menyelaraskan dependensi proyek di dalam lingkungan pengembangan lokal. Dalam proses ini, uv secara otomatis mendeteksi dan menggunakan CPython 3.10.19 untuk membuat sebuah lingkungan virtual baru pada direktori .venv. Perintah tersebut berhasil membuat sebuah virtual environment '.venv' (hidden) pada root folder, lalu memasang libraries yang dibutuhkan.

2. Client.py

Setelah instalasi tools pendukung selesai, kita mulai menyusun logika pada client.py. Pada file client.py, dilakukan proses pembuatan tanda tangan digital ganda menggunakan algoritma Ed25519 dan ECDSA, serta enkripsi pesan rahasia menggunakan kombinasi ECDH dan AES-GCM.

```
# TODO: Lengkapi proses-proses pembuatan private dan public key
# untuk users yang disimulasikan
priv_key = ...
pub_key = ...
```

Gambar 3. TODO pertama

TODO pertama, diminta untuk melengkapi proses-proses pembuatan private dan public key.

```
# TODO: Lengkapi proses-proses pembuatan private dan public key
# untuk users yang disimulasikan
#SECP256K1
priv_client_key = ec.generate_private_key(ec.SECP256K1(), default_backend())
pub_client_key = priv_client_key.public_key()
#ED25519
priv1 = ed25519.Ed25519PrivateKey.generate()
pub1 = priv1.public_key()
# Simpan ke file SECP256K1
with open("priv_client.pem", "wb") as f:
    f.write(priv_client_key.private_bytes(encoding=serialization.Encoding.PEM,
                                         format=serialization.PrivateFormat.PKCS8,
                                         encryption_algorithm=serialization.NoEncryption()))
with open("pub_client.pem", "wb") as f:
    f.write(pub_client_key.public_bytes(encoding=serialization.Encoding.PEM,
                                       format=serialization.PublicFormat.SubjectPublicKeyInfo))
# Simpan ke file ED25519
with open("priv1.pem", "wb") as f:
    f.write(priv1.private_bytes(encoding=serialization.Encoding.PEM,
                               format=serialization.PrivateFormat.PKCS8,
                               encryption_algorithm=serialization.NoEncryption()))
with open("pub1.pem", "wb") as f:
    f.write(pub1.public_bytes(encoding=serialization.Encoding.PEM,
                              format=serialization.PublicFormat.SubjectPublicKeyInfo))
```

Gambar 4. Kode TODO pertama

Untuk membuat private dan public key, menggunakan dua algoritma kriptografi yang berbeda, yaitu SECP256K1 dan ED25519 (algoritma Edwards-curve). Setelah itu melakukan serialisasi kunci ke dalam format PEM menggunakan standar PKCS8 untuk kunci privat dan SubjectPublicKeyInfo untuk kunci publik, kemudian menyimpannya ke dalam media penyimpanan lokal sebagai berkas .pem.

```
# TODO: Lengkapi proses-proses lain enkripsi simetrik (jika dibutuhkan)
# di mana pesan rahasia tersebut akan ditransmisikan
#
# Tulis code Anda di bawah ini
```

Gambar 5. TODO kedua

TODO kedua, diminta untuk melengkapi proses lain enkripsi simetrik (jika dibutuhkan). Dimana pesan rahasia tersebut akan ditransmisikan.

```
# Tulis code Anda di bawah ini
subject = issuer = x509.Name([x509.NameAttribute(NameOID.COUNTRY_NAME, "ID"),
                               x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, "Jawa Timur"),
                               x509.NameAttribute(NameOID.LOCALITY_NAME, "Surabaya"),
                               x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Ayam Kremes"),
                               x509.NameAttribute(NameOID.COMMON_NAME, SENDER_ID)])
cert = (x509.CertificateBuilder().subject_name(subject).issuer_name(issuer)
        .public_key(pub_client_key).serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.now(datetime.timezone.utc))
        .not_valid_after(datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(days=365))
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
        .sign(priv_client_key, hashes.SHA256(), default_backend()))
with open("client_certificate.pem", "wb") as f:
    f.write(cert.public_bytes(serialization.Encoding.PEM))
```

Gambar 6. Buat digital signature

Membuat *digital signature*, menetapkan informasi identitas seperti lokasi geografis dan nama organisasi, kemudian menandatangani sertifikat tersebut menggunakan kunci privat dengan algoritma SHA256. *Signature* ini dikonfigurasi dengan masa berlaku satu tahun dan disimpan dalam format `client_certificate.pem`, yang berfungsi sebagai kredensial autentikasi formal dalam arsitektur keamanan server.

```
# Pesan rahasia yang ingin dikirim ke server
message = b"Diam - diam yaa ini pesan rahasia."
signature_ed25519 = priv1.sign(message)
signature_ecdsa = priv_client_key.sign(message, ec.ECDSA(hashes.SHA256()))
print("Signature ED25519:", len(signature_ed25519))
print("Signature ECDSA:", len(signature_ecdsa))
```

Gambar 7. Implementasi digital signature

Menghasilkan *digital signature* dari sebuah pesan rahasia menggunakan dua algoritma kriptografi yang berbeda. Dengan algoritma Ed25519 (`priv1`) dan kedua dengan algoritma ECDSA yang dikombinasikan dengan hashing SHA-256 (`priv_client_key`). Tujuannya untuk memastikan integritas dan keaslian pesan, sehingga penerima tahu bahwa pesan tersebut benar-benar dikirim oleh pemilik kunci privat dan belum diubah oleh pihak lain.

```
def derive_shared_key(priv_ec, peer_pub):
    secret = priv_ec.exchange(ec.ECDH(), peer_pub)
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                info=b"ecdh-ecies", backend=default_backend()).derive(secret)
```

Gambar 8. Pertukaran kunci

Fungsi ini untuk melakukan pertukaran kunci aman menggunakan protokol kriptografi asimetris. Fungsi `derive_shared_key` menerima dua parameter, yaitu kunci privat milik sendiri (`priv_ec`) dan kunci publik milik rekan komunikasi (`peer_pub`). Program menggunakan metode ECDH (*Elliptic Curve Diffie-Hellman*) untuk menghasilkan sebuah nilai rahasia bersama dari kombinasi kedua kunci tersebut. Nilai rahasia mentah tersebut tidak langsung

digunakan, melainkan diproses melalui HKDF dengan algoritma SHA-256 untuk menghasilkan kunci akhir sepanjang 32 byte.

Tujuannya untuk memastikan bahwa kedua pihak dapat memiliki kunci enkripsi yang sama tanpa pernah mengirimkan kunci tersebut melalui jaringan, yang biasanya digunakan dalam sistem ECIES (Elliptic Curve Integrated Encryption Scheme).

```
def encrypt_ecies(recipient_pub, plaintext, sender_id, receiver_id):
    eph_priv = ec.generate_private_key(ec.SECP256K1(), default_backend())
    eph_pub = eph_priv.public_key()
    key = derive_shared_key(eph_priv, recipient_pub)
    aesgcm = AESGCM(key)
    nonce = os.urandom(12)
    aad = f"{sender_id}|{receiver_id}".encode()
    ciphertext = aesgcm.encrypt(nonce, plaintext, aad)
    eph_pub_pem = eph_pub.public_bytes(serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)
    return eph_pub_pem, nonce, ciphertext
```

Gambar 9. Proses enkripsi

proses enkripsi dengan membuat pasangan kunci sementara (ephemeral key) menggunakan kurva SECP256K1. Kunci sementara tersebut digabungkan dengan kunci publik penerima untuk menghasilkan kunci simetris melalui fungsi derivasi yang telah dibuat sebelumnya. *Plaintext* dienkripsi menggunakan algoritma AES-GCM, hasil dari fungsi ini mengembalikan kunci publik sementara, nonce (angka acak sekali pakai), dan teks terenkripsi (ciphertext). Kunci enkripsi yang digunakan selalu berubah setiap kali pesan dikirim, sehingga meskipun satu pesan terbongkar, pesan lainnya tetap terlindungi.

```
with open("punkhazard-keys/pub.pem", "rb") as f:
    server_public_key = serialization.load_pem_public_key(f.read())

eph_pub_pem, nonce, ciphertext = encrypt_ecies(
    server_public_key,
    message,
    SENDER_ID,
    RECEIVER_ID,
)
```

Gambar 10. Fungsi proses enkripsi

Program memuat kunci publik server dari sebuah file eksternal bernama *pub.pem*. Seluruh komponen tersebut kemudian dipanggil untuk mengenkripsi message sebelum dikirimkan ke server, memastikan hanya pemilik kunci privat server yang dapat membacanya.

```

with open("pub_client.pem", "rb") as f:
    client_pub_pem = f.read()

with open("client_certificate.pem", "rb") as f:
    cert_pem = f.read()

payload = {
    "sender": SENDER_ID,
    "receiver": RECEIVER_ID,
    "certificate": b64(cert_pem),
    "client_public_key": b64(client_pub_pem),
    "ephemeral_public_key": b64(eph_pub_pem),
    "nonce": b64(nonce),
    "ciphertext": b64(ciphertext),
    "signature_ed25519": b64(signature_ed25519),
    "signature_ecdsa": b64(signature_ecdsa),
}

print("Payload siap dikirim ke server:")
payload_json = json.dumps(payload, indent=4)
print(payload_json)

```

Gambar 11. Fungsi proses enkripsi

Seluruh komponen keamanan, kunci publik klien, sertifikat digital, kunci publik sementara (ephemeral), nonce, ciphertext, serta dua jenis tanda tangan digital (Ed25519 dan ECDSA) dikumpulkan ke dalam satu struktur data JSON. Sebelum dikirim, semua data biner (seperti kunci dan tanda tangan) diubah ke dalam format Base64 agar aman saat ditransmisikan melalui protokol teks seperti HTTP. Program tersebut memastikan bahwa pesan tidak hanya tidak dapat dibaca oleh pihak ketiga, tetapi juga terverifikasi berasal dari pengirim yang sah dan tidak mengalami perubahan selama perjalanan.

3. api.py

```

from urllib import request
from fastapi import FastAPI, HTTPException, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from typing import Optional, List
from pydantic import BaseModel
import os
from datetime import datetime
from contextlib import contextmanager
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec, ed25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography import x509
from cryptography.hazmat.backends import default_backend
import base64

```

Import library yang dibutuhkan untuk membangun API web (FastAPI) yang memiliki fitur keamanan tingkat tinggi, khususnya dalam menangani kriptografi (seperti enkripsi AESGCM, tanda tangan digital Ed25519, dan sertifikat X.509) serta pengelolaan unggahan berkas.

```
app = FastAPI(title="Security Service", version="1.0.0")
AUTHORIZED_USERS = ["client-ayam-kremes", "server-punkhazard"]
```

untuk mendefinisikan daftar kontrol akses (Access Control List) yang berisi identitas pengguna atau klien yang diizinkan untuk berinteraksi dengan layanan "Security Service". Dengan menyimpan nama unik seperti client-ayam-kremes dan server-punkhazard dalam variabel AUTHORIZED_USERS, sistem memiliki referensi statis untuk melakukan validasi otorisasi pada setiap permintaan yang masuk ke API.

```
# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

untuk mengonfigurasi infrastruktur dasar layanan API yang aman dengan menginisialisasi aplikasi FastAPI bernama "Security Service" sekaligus menetapkan kebijakan akses dan identitas server. Pengaturan CORS middleware dengan parameter terbuka (*) memungkinkan aplikasi menerima permintaan dari berbagai domain, sementara daftar AUTHORIZED_USERS menyediakan basis data statis untuk kontrol akses klien. Selain itu, pemuatan kunci privat melalui serialization.load_pem_private_key memberikan kemampuan kriptografis kepada server untuk melakukan autentikasi dan enkripsi data sensitif.

```
# Load server private key
with open("punkhazard-keys/priv.pem", "rb") as f:
    SERVER_PRIVATE_KEY = serialization.load_pem_private_key(f.read(), password=None, backend=default_backend())
```

untuk membangun fondasi layanan API yang aman dengan menginisialisasi aplikasi FastAPI bernama "Security Service" sekaligus menetapkan kebijakan keamanan dasar.

```
# Simulasi token user
USER_TOKENS = {
    "client-ayam-kremes": "token123456789",
    "server-punkhazard": "token987654321"
}
```

untuk membangun fondasi layanan API (FastAPI) yang aman dengan mengintegrasikan sistem otentikasi berbasis token dan kontrol akses yang ketat.

```

# Helper functions
def b64encode(data: bytes) -> str:
    | return base64.b64encode(data).decode()

def b64decode(data: str) -> bytes:
    | return base64.b64decode(data.encode())

def derive_shared_key(priv, peer_pub):
    | secret = priv.exchange(ec.ECDH(), peer_pub)
    | return HKDF(
    |     algorithm=hashes.SHA256(),
    |     length=32,
    |     salt=None,
    |     info=b"ecies-ecdh",
    |     backend=default_backend()
    | ).derive(secret)

def verify_token(username: str, authorization: str):
    | if not authorization or not authorization.startswith("Bearer "):
    |     | raise HTTPException(status_code=403, detail="Missing or invalid Bearer token")
    | token = authorization.split()[1]
    | expected = USER_TOKENS.get(username)
    | if token != expected:
    |     | raise HTTPException(status_code=403, detail="Unauthorized token")

```

Berfungsi sebagai infrastruktur keamanan lengkap pada FastAPI yang mencakup aspek konektivitas, otentikasi, dan perlindungan data kriptografis. Sistem ini mengaktifkan CORS middleware untuk mengelola akses lintas domain, memverifikasi identitas pengguna melalui pencocokan token Bearer dengan daftar USER_TOKENS yang telah ditentukan, serta memuat kunci privat server untuk mendukung operasi enkripsi tingkat lanjut. Selain itu, terdapat fungsi pembantu (helper functions) untuk konversi format Base64 dan mekanisme Diffie-Hellman (ECDH) guna menghasilkan kunci rahasia bersama yang aman bagi pertukaran data sensitif.

```

# Health check
@app.get("/health")
async def health_check():
    | return {"status": "Security Service is running", "timestamp": datetime.now().isoformat()}

```

untuk membangun infrastruktur API keamanan terintegrasi yang mencakup manajemen akses, otentikasi, dan operasional layanan. Fungsi pembantu untuk pertukaran kunci rahasia menggunakan metode ECDH serta endpoint /health yang berperan sebagai mekanisme health check untuk memantau status operasional dan ketersediaan layanan secara real-time.

```

# Index
@app.get("/")
async def get_index():
    | return {"message": "Hello world! Visit http://localhost:8080/docs for API UI."}

```

Untuk membangun sebuah layanan mikro keamanan terintegrasi menggunakan FastAPI yang mencakup aspek konektivitas, otentikasi, dan perlindungan data kriptografis.

```
# Upload PDF
@app.post("/upload-pdf")
async def upload_pdf(file: UploadFile = File(...)):
    os.makedirs("uploads", exist_ok=True)
    try:
        contents = await file.read()
        with open(f"uploads/{file.filename}", "wb") as f:
            f.write(contents)
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Upload failed: {str(e)}")
    return {"message": "File uploaded!", "filename": file.filename, "content-type": file.content_type}
```

berfungsi untuk menerima unggahan file PDF dari pengguna dan menyimpannya secara lokal ke dalam direktori server. Kode ini bekerja dengan cara membuat folder penyimpanan otomatis, membaca isi file secara asinkron guna menjaga performa aplikasi, lalu menulis ulang data tersebut ke dalam sistem penyimpanan permanen sebelum mengembalikan respons sukses kepada pengguna.

```
# Store public key
@app.post("/store")
async def store_pubkey(username: str, algo: str, expected_hash: str,
                       pubkey: UploadFile = File(...), authorization: str = Header(None)):
    verify_token(username, authorization)

    if username not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403, detail=f"User '{username}' tidak terdaftar.")
    try:
        data = await pubkey.read()
        actual_hash = hashlib.sha256(data).hexdigest()
        if actual_hash != expected_hash:
            return {"message": "Integrity Check Gagal! File rusak atau dimodifikasi.", "success": False}

        user_dir = f"pubkeys/{username}"
        os.makedirs(user_dir, exist_ok=True)
        fname = "pub.pem" if algo.lower() == "ecdsa" else "pub19.pem"
        with open(f"{user_dir}/{fname}", "wb") as f:
            f.write(data)

        return {"message": "Public key stored successfully.", "user": username, "success": True, "hash_calculated": actual_hash}
    except Exception as e:
        return {"message": f"Error: {str(e)}", "user": username, "success": False}
```

Berfungsi untuk menerima, memvalidasi, dan menyimpan kunci publik (public key) milik pengguna ke dalam server secara aman. Prosesnya mencakup verifikasi token otorisasi, pengecekan integritas file menggunakan hash SHA-256 untuk memastikan data tidak rusak atau dimodifikasi, serta pengorganisasian penyimpanan file berdasarkan nama pengguna dan jenis algoritma kunci yang digunakan.

```

# Verify message signature
@app.post("/verify")
async def verify(username: str, message: str, signature: str, algo: str, message_hash: str,
                 | authorization: str = Header(None)):
    verify_token(username, authorization)

    if username not in AUTHORIZED_USERS:
        | raise HTTPException(status_code=403, detail=f"User '{username}' tidak memiliki izin akses.")

    calculated_hash = hashlib.sha256(message.encode()).hexdigest()
    if calculated_hash != message_hash:
        | return {"message": "Integrity Check Gagal! Pesan telah dimodifikasi.",
        |         "user": username, "valid": False, "integrity_check": "Failed",
        |         "calculated_hash": calculated_hash}

    valid = False
    msg = None
    try:
        | key_file = f"pubkeys/{username}/pub.pem" if algo.lower() == "ecdsa" else f"pubkeys/{username}/pub19.pem"
        | with open(key_file, "rb") as f:
        |     pubkey = serialization.load_pem_public_key(f.read())
        | sig_bytes = b64decode(signature)

        | if algo.lower() == "ecdsa":
        |     pubkey.verify(sig_bytes, message.encode(), ec.ECDSA(hashes.SHA256()))
        | elif algo.lower() == "ed25519":
        |     pubkey.verify(sig_bytes, message.encode())
        | else:
        |     return {"message": "Unknown algorithm", "user": username, "valid": False}

        | msg = "Signature VALID"
        | valid = True
    except Exception:
        | msg = "Signature INVALID"
        | valid = False

    return {"message": msg, "user": username, "valid": valid, "algo": algo.lower(),
           | "integrity_check": "Success" if valid else "Failed", "calculated_hash": calculated_hash}

```

Berfungsi untuk membuktikan keaslian sebuah pesan dengan mencocokkan tanda tangan digital menggunakan algoritma ECDSA atau Ed25519. Secara keseluruhan, rangkaian kode ini menjamin bahwa hanya pengguna terdaftar yang dapat mengirim pesan dan memastikan data yang diterima benar-benar asli serta tidak dimodifikasi oleh pihak ketiga.

```

# Verify PDF signature
@app.post("/verify-pdf")
async def verify_pdf(username: str, algo: str, pdf_hash: str, signature_pdf: str,
                    | authorization: str = Header(None)):
    verify_token(username, authorization)

    key_file = f"pubkeys/{username}/pub.pem" if algo=="ecdsa" else f"pubkeys/{username}/pub19.pem"
    with open(key_file, "rb") as f:
        | pubkey = serialization.load_pem_public_key(f.read())

    sig_bytes = b64decode(signature_pdf)
    pdf_bytes = b64decode(pdf_hash)

    try:
        | if algo == "ecdsa":
        |     pubkey.verify(sig_bytes, pdf_bytes, ec.ECDSA(hashes.SHA256()))
        | elif algo == "ed25519":
        |     pubkey.verify(sig_bytes, pdf_bytes)
        | else:
        |     return {"message": "Unknown algorithm", "valid": False}

        | msg = "Signature VALID"
        | valid = True
        | return {"message": msg, "user": username, "valid": valid,
        |         "integrity_check": "Success" if valid else "Failed",
        |         "algo": algo, "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}
    except:
        | return {"message": msg, "user": username, "valid": False,
        |         "integrity_check": "Failed", "algo": algo,
        |         "calculated_hash": hashlib.sha256(pdf_bytes).hexdigest()}

```

berfungsi untuk mengelola otentikasi data melalui penyimpanan kunci publik dan verifikasi tanda tangan digital baik untuk pesan teks maupun dokumen PDF. Sistem ini menyimpan public key pengguna dengan validasi integritas SHA-256, kemudian menggunakan kunci tersebut untuk membuktikan keaslian data dan identitas pengirim menggunakan algoritma kriptografi ECDSA atau Ed25519.

```
# Relay secure message
@app.post("/relay")
async def relay(sender: str, receiver: str, ephemeral_public_key: str, nonce: str, ciphertext: str,
                signature: str = None, algo: str = None, ciphertext_hash: str = None,
                authorization: str = Header(None)):
    verify_token(sender, authorization)

    if sender not in AUTHORIZED_USERS or receiver not in AUTHORIZED_USERS:
        raise HTTPException(status_code=403, detail="Sender atau Receiver tidak terdaftar.")

    if ciphertext_hash and hashlib.sha256(ciphertext.encode()).hexdigest() != ciphertext_hash:
        return {"message": "Integrity Check Gagal! Ciphertext rusak.", "success": False}

    try:
        eph_pub = serialization.load_pem_public_key(b64decode(ephemeral_public_key))
        nonce_b = b64decode(nonce)
        cipher_b = b64decode(ciphertext)

        key = derive_shared_key(SERVER_PRIVATE_KEY, eph_pub)
        aesgcm = AESGCM(key)
        aad = f"{sender}|{receiver}".encode()
        plaintext = aesgcm.decrypt(nonce_b, cipher_b, aad)

        if signature:
            key_file = f"pubkeys/{sender}/pub.pem" if algo.lower() == "ecdsa" else f"pubkeys/{sender}/pub19.pem"
            with open(key_file, "rb") as f:
                pubkey = serialization.load_pem_public_key(f.read())
            sig_bytes = b64decode(signature)
            if algo.lower() == "ecdsa":
                pubkey.verify(sig_bytes, plaintext, ec.ECDSA(hashes.SHA256()))
            else:
                pubkey.verify(sig_bytes, plaintext)

            os.makedirs("messages", exist_ok=True)
            with open(f"messages/{receiver}.txt", "a") as f:
                f.write(f"From {sender}: {plaintext.decode()}\n")

        return {"message": "Secure message relayed", "from": sender, "to": receiver, "success": True}
    except Exception as e:
        return {"message": f"Relay failed: {str(e)}", "success": False}
```

Berfungsi untuk mengelola siklus hidup komunikasi terenkripsi dan tervalidasi antar pengguna. Secara spesifik, kode tersebut mengintegrasikan fungsi penyimpanan kunci publik dengan pengecekan integritas SHA-256, verifikasi tanda tangan digital untuk pesan teks dan dokumen PDF guna menjamin keaslian data, serta fitur pesan aman (relay) yang menggunakan enkripsi AES-GCM dan derivasi kunci bersama untuk melindungi kerahasiaan isi pesan. Kegunaan utama dari sistem tersebut adalah untuk membangun infrastruktur pertukaran informasi yang memenuhi standar keamanan informasi, yaitu memastikan data tetap rahasia (confidentiality), tidak diubah oleh pihak luar (integrity), dan benar-benar berasal dari pengirim yang sah (authenticity).

Alur Algoritma

1. Tahap persiapan dan instalasi

Instalasi uv, memasang biner uv untuk manajemen eksekusi yang cepat. uv sync, menyelaraskan seluruh dependensi (seperti fastapi, cryptography, dan uvicorn) ke dalam virtual environment .venv.

2. Alur kerja sisi Client

Klien bertanggung jawab untuk menyiapkan identitas keamanan dan mengenkripsi pesan sebelum dikirim ke server.

- Membuat pasangan kunci privat dan publik menggunakan algoritma SECP256K1 (untuk enkripsi) dan ED25519 (untuk tanda tangan digital).
- Menyusun sertifikat digital (format X.509) yang berisi identitas pengirim dan ditandatangani dengan kunci privat.
- Penandatanganan Pesan: Pesan rahasia ditandatangani dua kali (ganda) menggunakan Ed25519 dan ECDSA untuk menjamin keaslian (otentikasi).
- Proses Enkripsi (ECIES):
Membuat kunci sementara (Ephemeral Key). Melakukan pertukaran kunci ECDH dengan kunci publik server untuk menghasilkan rahasia bersama. Memperkuat rahasia tersebut menggunakan HKDF (SHA-256) menjadi kunci simetris 32-byte. Mengenkripsi pesan asli menggunakan AES-GCM dengan tambahan nonce dan data autentikasi.
- Mengonversi semua data (kunci, ciphertext, signature) ke format Base64 dan membungkusnya dalam JSON untuk dikirim melalui API.

3. Alur kerja sisi Server (API)

Server bertindak sebagai pusat pengamanan dan penyedia layanan (Security Service).

- Endpoint store
Menerima kunci publik klien. Melakukan Integrity Check dengan membandingkan hash file yang diterima dengan expected_hash untuk memastikan file tidak dimodifikasi saat pengiriman.
- Endpoint verify
Memvalidasi tanda tangan digital pesan menggunakan kunci publik yang telah disimpan. Memastikan isi pesan (integrity) tidak berubah menggunakan perbandingan hash SHA-256
- Endpoint relay
Menerima payload terenkripsi. Melakukan derivasi kunci (derive_shared_key) untuk mendapatkan kunci dekripsi yang sama. Mendekripsi pesan menggunakan AES-GCM. Menyimpan pesan yang telah didekripsi ke dalam file teks di server jika verifikasi identitas berhasil.
- Endpoint upload-pdf dan verify-pdf
Mengelola penyimpanan dokumen PDF dan melakukan verifikasi tanda tangan digital khusus untuk file dokumen guna menjamin integritas data PDF tersebut.

Link Github: [UAS-KID-KEL04](#)